

模型量化与推理优化

大模型推理是 " 显存 " 与 " 算力 " 的双重博弈——量化压缩显存，注意力优化榨干算力，二者协同让百亿参数模型在 GPU 上成为可能。

模型量化与推理优化

大模型推理是 " 显存 " 与 " 算力 " 的双重博弈——量化压缩显存，注意力优化榨干算力，二者协同让百亿参数模型在 GPU 上成为可能。

- - -

为什么需要推理优化

大语言模型（LLM）在推理阶段面临两大核心瓶颈：显存占用与计算延迟。一个 70B 参数模型，仅权重存储就需要约 140GB 显存，远超单张消费级 GPU（如 RTX 4090 的 24GB）。同时，注意力机制的 O(1) 复杂度与 KV Cache 解码的特性，导致计算密集度低、显存带宽成为瓶颈。

模型规模 → 显存墙：70B FP16 ≈ 140GB，单卡装不下

自回归生成 → 带宽墙：每生成1个token，需读取全部权重 + KV Cache

长上下文 → KV Cache 膨胀：32K 上下文的 KV Cache 可达数 GB

推理优化的目标是在不显著损失精度的前提下，降低显存占用、提升吞吐量（tokens/s）、降低首字延迟（TPO T）。其价值贯穿从云端服务到边缘部署的全场景：

场景 瓶颈 优化重点

云端高并发服务 吞吐量、成本 批处理、Paged Attention、量化

边缘 / 本地部署 显存、延迟 量化到 INT4、llama.cpp/Ollama

长上下文应用 KV Cache 显存 Paged Attention、KV Cache 量化

实时对话 首 token 延迟 投机解码、Prefix Cache

- - -

量化技术（INT8、INT4、GPTQ、AWQ、GGUF）

量化（Quantization）

是将模型权重从高精度浮点（FP16 / FP32）压缩为低精度整数（INT8 / INT4）的技术，能近

1 . 量化精度等级

精度 字节 / 参数 70B 显存 适用场景

FP32 4 280GB 训练、精度基准

FP16 / BF16 2 140GB 标准推理

INT8 1 70GB 服务端量化

INT4 0.5 35GB 单卡 / 本地部署

2. 量化方法分类

训练后量化 (PTQ, Post-Training Quantization)
无需重新训练, 直接对已训模型转换, 工程友好, 是主流路线。

量化感知训练 (QAT, Quantization-Aware Training)
在训练中模拟量化误差, 精度更高但成本大, 较少用于 LLM。

3. 主流量化算法

- GPTQ: 基于二阶 Hessian 信息的逐层量化, 逐列补偿误差。INT4 精度优秀, 但量化过程较慢, 适合服务端部署。

- AWQ (Activation-aware Weight Quantization): 发现 " 激活幅度大的通道 " 的权重对精度更关键, 对其保留高精度 (按比例缩放) INT4。精度与 GPTQ 相当, 量化更快, 推理更友好。

- GGUF (前 GGML): llama.cpp 生态的量化格式, 支持 K-quants (Q4K/M) CPU / 混合架构上表现优异, 是本地部署的事实标准。

- BitsAndBytes (bnb): 8-bit / 4-bit 量化, 集成于 HuggingFace transformers, 零代码即可加载, 适合快速实验, 但推理速度不如专用量化。

AWQ 量化模型加载 (vLLM)

```
from vllm import LLM
llm = LLM(model="TheBloke/Llama-2-13B-AWQ", quantization="awq")
```

BitsAndBytes 4-bit 加载 (HuggingFace)

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_computed_type="bfloat16",
    bnb_4bit_used_double_quant=True,
)
```

```
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b-hf",
quantization_config=bnb_config)
```

选型建议：服务端追求性能选 AWQ / GPTQ + vLLM；本地 CPU / 混合部署选 GGUF + llama.cpp / Ollama；快速实验选 BitsAndBytes。

- - -

KV Cache 优化

自回归生成时，每生成一个新 token 都要重新计算所有历史 token 的 Key / Value。Cache 将历史层的 K、V 张量缓存复用，把计算复杂度从 $O(n^2)$ 降到 $O(n)$ ，是 LLM 推理加速的关键。

但 KV Cache 会随序列长度线性增长：以 Llama-2-7B、32K 上下文为例，单条请求需 1.5GB，多并发时显存碎片化严重。

优化策略

1. Paged Attention：借鉴操作系统虚拟内存的分页机制，将 KV Cache 划分为固定大小的 "页" (block)，按需分配，消除碎片，提升显存利用率与并发吞吐。是 vLLM 的核心优化。
2. KV Cache 量化：将 FP16 的 K/V 压缩为 INT8 / INT4，显存减半甚至更低。
3. Prefix Cache：对相同系统提示 / 前缀的请求复用 KV Cache，避免重复计算，显著提升长上下文性能。
4. Sliding Window Attention：只缓存最近 N 层窗口的 KV，牺牲长程依赖换取短程性能。

vLLM 默认启用 PagedAttention，通过参数调优

```
from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Llama-2-7b",
    enable_prefix_caching=True, # 前缀缓存
    gpu_memory_utilization=0.9, # 显存利用率
    max_model_len=8192,
)
```

- - -

注意力优化 (Flash Attention、Paged Attention、Ring Attention)

标准 Attention 的核心公式 $\text{softmax}(QK^T / \sqrt{d})V$ 在长序列下显存与计算均呈平方增长，是推理与训练的主要瓶颈。

1. Flash Attention

通过分块计算 (Tiling) 与算子融合 (Kernel Fusion), 将 QKV 分块加载到 C HBM 中实例化完整的 $n \times n$ 注意力矩阵。Flash Attention v2 进一步优化并行度优化。它在不改变数学结果的前提下, 将显存从 $O(n^2)$ 降到 $O(n)$, 并大幅提速。

2. Paged Attention

如前所述, 针对 KV Cache 的分页管理, 解决并发场景下的显存碎片, 是 vLLM 高吞吐的关键。

3. Ring Attention / 分布式注意力

将长序列切分到多张 GPU, 每张卡持有部分 KV, 通过环形通信 (Ring All-Gather) 在并协作计算注意力。使单机无法装下的超长上下文 (如 1M token) 成为可能。

Flash Attention: 算子级优化, 单卡内降低显存、加速计算

Paged Attention: 系统级优化, 多请求并发下管理 KV Cache

Ring Attention: 分布式优化, 跨卡切分超长序列

- - -

投机解码与 Medusa

标准自回归解码每步只生成 1 个 token, 且每次都要把全部权重和 KV Cache 读一遍, 算力利用率低、显存带宽受限。

1. 投机解码 (Speculative Decoding)

用一个小的草稿模型 (Draft Model) 快速生成若干候选 token, 再用大模型并行验证 (1 token)。被接受的 token 一次性产出, 被拒绝的位置回退重算。由于大模型验证多 token token 接近, 整体可加速 2 - 3 倍, 且输出与原模型完全一致 (无损)。

2. Medusa

无需单独草稿模型, 在大模型上额外训练多个 "头" 预测后续多个位置的 token, 构建树形候选并并行验证。工程更简单, 加速 2 倍左右。

投机解码示意 (vLLM 支持)

```
from vllm import LLM
llm = LLM(
```

```
model = "meta-llama/Llama-2-70b",
speculativemodel = "meta-llama/Llama-2-7b", # 草稿模型
numspeculativetokens = 5,
)
```

投机解码在数学 / 代码等确定性强的任务上加速显著，在创意写作等高温采样场景收益较小。

- - -

模型蒸馏与剪枝

1. 模型蒸馏 (Distillation)

将大模型（教师）的能力迁移到小模型（学生）。学生通过模仿教师的输出分布（软标签）或中间层特征进行学习（97% 能力）、Alpaca / Vicuna（用 GPT-4 输出蒸馏开源模型）。

蒸馏损失（简化）

```
def distillloss(studentlogits, teacherlogits, labels):
    softloss = kldiv(logsoftmax(studentlogits/T), logsoftmax(teacherlogits/T))
    hardloss = crossentropy(studentlogits, labels)
    return alpha * softloss + (1 - alpha) * hardloss
```

2. 剪枝 (Pruning)

移除对输出贡献小的参数：结构化剪枝 删除整个注意力头、层或通道，硬件友好；非结构化剪枝 将个别权重置零，需稀疏计算支持。LLM 中剪枝应用较谨慎，因为大模型对参数删除敏感，常用的是层删除 (Layer Drop) 的轻量剪枝。

- - -

推理框架对比 (vLLM、TGI、TensorRT-LLM、llama.cpp、Ollama)

框架 开发方 核心优势 适用场景

框架	开发方	核心优势	适用场景
vLLM	UC Berkeley	PagedAttention、高吞吐、易用	云端生产服务（最主流推理框架）
TGI	HuggingFace	与 HF 生态深度集成、协议完善	HF 生态用户
TensorRT-LLM	NVIDIA	极致 GPU 性能、A100 / H100 优化	追求极致性能
llama.cpp	Georgi Gerganov	CPU / 混合推理、GGUF 量化、轻量	本地部署
Ollama	Ollama	一行命令运行模型、API 友好	开发者本地体验、原型

vLLM 服务端启动

命令行: `python -m vllm.entrypoints.openai.api_server`

Python 调用

```
from vllm import LLM, SamplingParams
llm = LLM(model="meta-llama/llama-2-7b")
outputs = llm.generate(["解释一下什么是 RAG"], SamplingParams(
    max_tokens=200))
```

Ollama 本地运行 (极简)

`ollama pull llama3`

`ollama run llama3 "用一句话解释 RAG"`

部署架构选型

不同场景对显存、吞吐、延迟、成本的要求差异很大, 应分层选型:

高并发云端服务

- vLLM / TGI + AWQ 量化 + PagedAttention + 投机解码
- 多卡张量并行 (Tensor Parallel) + 多副本负载均衡

企业私有化部署

- vLLM + GPTQ / AWQ (单卡或少量卡)
- 注意数据合规, 模型与数据均不出内网

本地 / 边缘部署

- llama.cpp / Ollama + GGUF INT4 量化
- CPU 推理或 CPU+GPU 混合, 显存不足时 offload 到内存

长上下文场景

- Ring Attention 分布式 + KV Cache 量化
- 配合 Prefix Cache 复用系统提示

选型决策清单

- 1 . 显存是否装得下？ 装不下 → 量化（INT4）或换更小模型 / 分布式。
- 2 . 并发有多高？ 高并发 → vLLM + PagedAttention + 连续批处理。
- 3 . 延迟要求多严？ 实时对话 → 投机解码 + Prefix Cache 降首字延迟。
- 4 . 是否 NVIDIA 生态？ 是且追求极致 → TensorRT-LLM；否 → vLLM。
- 5 . 是否需要离线 / 边缘？ 是 → llama.cpp + GGUF。

推理优化是一个系统工程：量化降显存、注意力优化提算力、批处理与 PagedAttention 提吞吐、投机解码降延迟。单点优化收益有限，组合优化才能释放大模型的全部潜能。

关键词：量化， INT8 ， GPTQ ， AWQ ， Flash Attention ， vLLM ， 燕